

September 2026

AI-Powered Fraud Detection

Fraud Detection Initiative

Technical Presentation

Emmanuel Tan Jr.



Problem Statement

Manual review of transactions and one-size-fits all tagging of fraudulent transactions cannot catch-up on the growing number of volume of digital and mobile-banking transactions. The goal of this new model is to flag high-risk transactions for further review before the actual fraud payout happens.

Task Type Binary classification (fraud vs legitimate transaction)

Primary metrics PR-AUC and Recall on the fraud class, not accuracy due to the 0.129% fraud rate

Business KPI Dollar amount of the fraud case prevented, cost of reviewing false-positive cases

Success Criteria

- PR-AUC over ROC-AUC
- Recall weighted heavily – missed fraud costs more than a false-positive tagging
- Precision to assess protected customer accounts

Synthetic Dataset – High-Level Details

6.36M

Transaction volume

0.129%

Fraud rate tagged

11

Data columns

Class imbalance – 6,354,407 legitimate vs 8,213 fraud transactions

A model that predicts legitimate transactions would be 99.87% accurate and will not catch fraud.

Exploratory Data Analysis

Fraud Rate per transaction type

0.77%

TRANSFER

0.18%

CASH_OUT

0%

PAYMENT

0%

CASH_IN

0%

DEBIT

Fraud lies on two types based on transaction structure

TRANSFER and CASH_OUT are the data types where fraud transactions happen as it can drain a customer account. This is helpful in building the model assumptions

Feature Engineering and Selection

$\text{errorBalanceOrig} / \text{errorBalanceDest}$

This yields balance-mismatch that dominates fraud signals

isMerchantDest

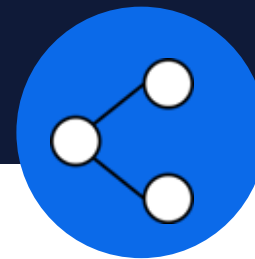
This is used to flag merchant destination accounts. No fraudulent tagging based on transaction structure.

Modelling Approach



Logistic Regression

Linear baseline, scaled features, class_weight = balanced



Random Forest

100 trees, max_depth=10, class_weight=balanced



XGBoost

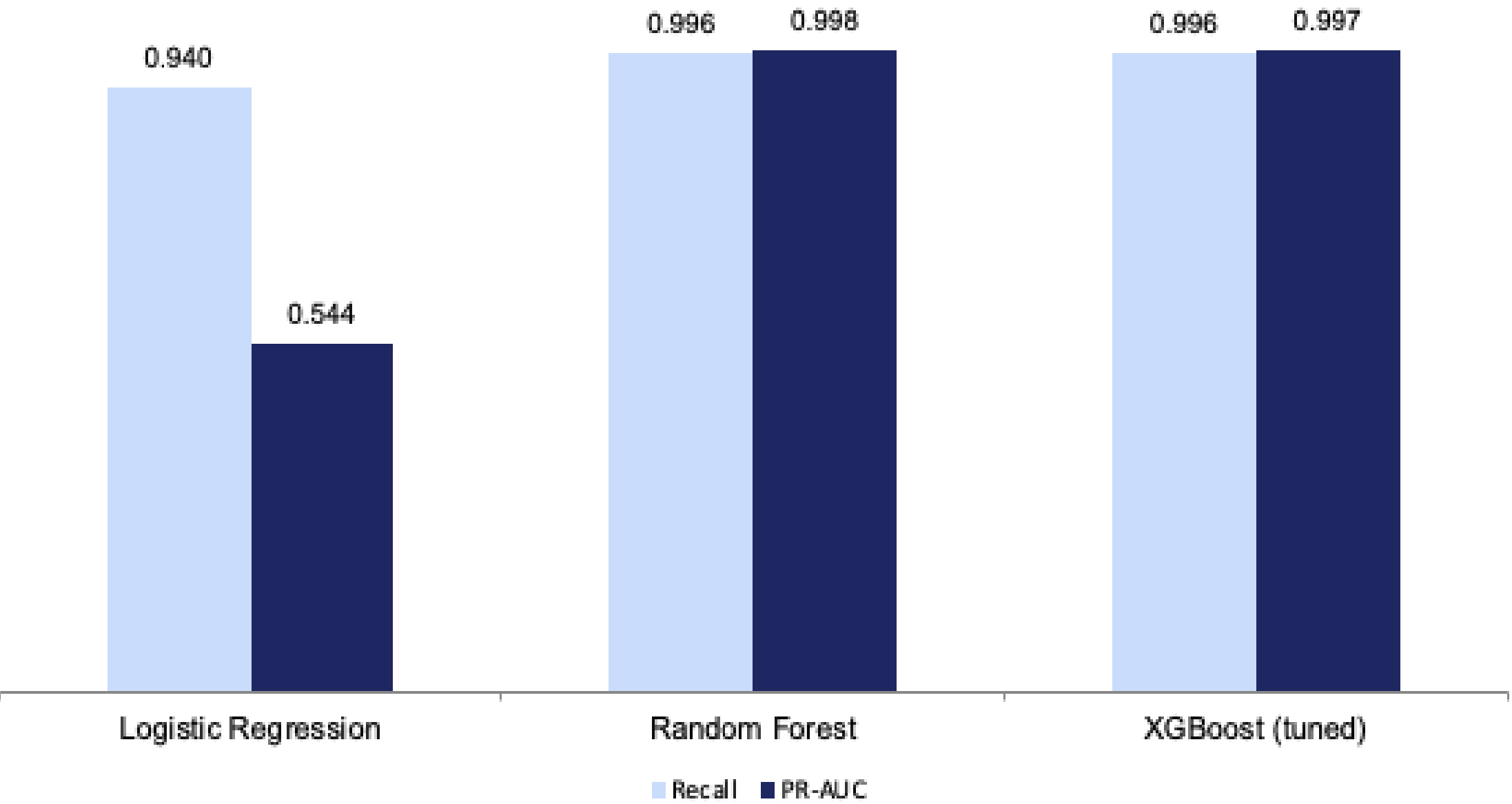
Tuned scale_pos_weight

Class weighting used

Oversampling the minority class would synthesize ~4.4M fake fraud on top of the ~5,750 real ones, this will incur computational wastage. Stratified split of 70/30 preserves real 0.129% on both sets

Model Comparison

New Model



Existing rule-based system

- 16/8,213 fraud cases based on rule-based system via **isFlaggedFraud** tagging

Random Forest throughput

- 2,455 / 2,464 caught by Random Forest with zero false positive tagging

Critical Findings

Default hyperparameters check

- XGBoost default weightage adjustment to correct the default 774 to 1.

scale_pos_weight = 774

PR-AUC 0.158

Precision 0.159

scale_pos_weight = 1

PR-AUC 0.997

Precision 0.999

Near-perfect score

- Dataset forces **newBalanceOrig** to exactly 0 on fraudulent transfers. This makes a mismatch feature as an indicator.

Full feature set (incl. balance-error terms)

PR-AUC 0.997

Without two engineered error terms

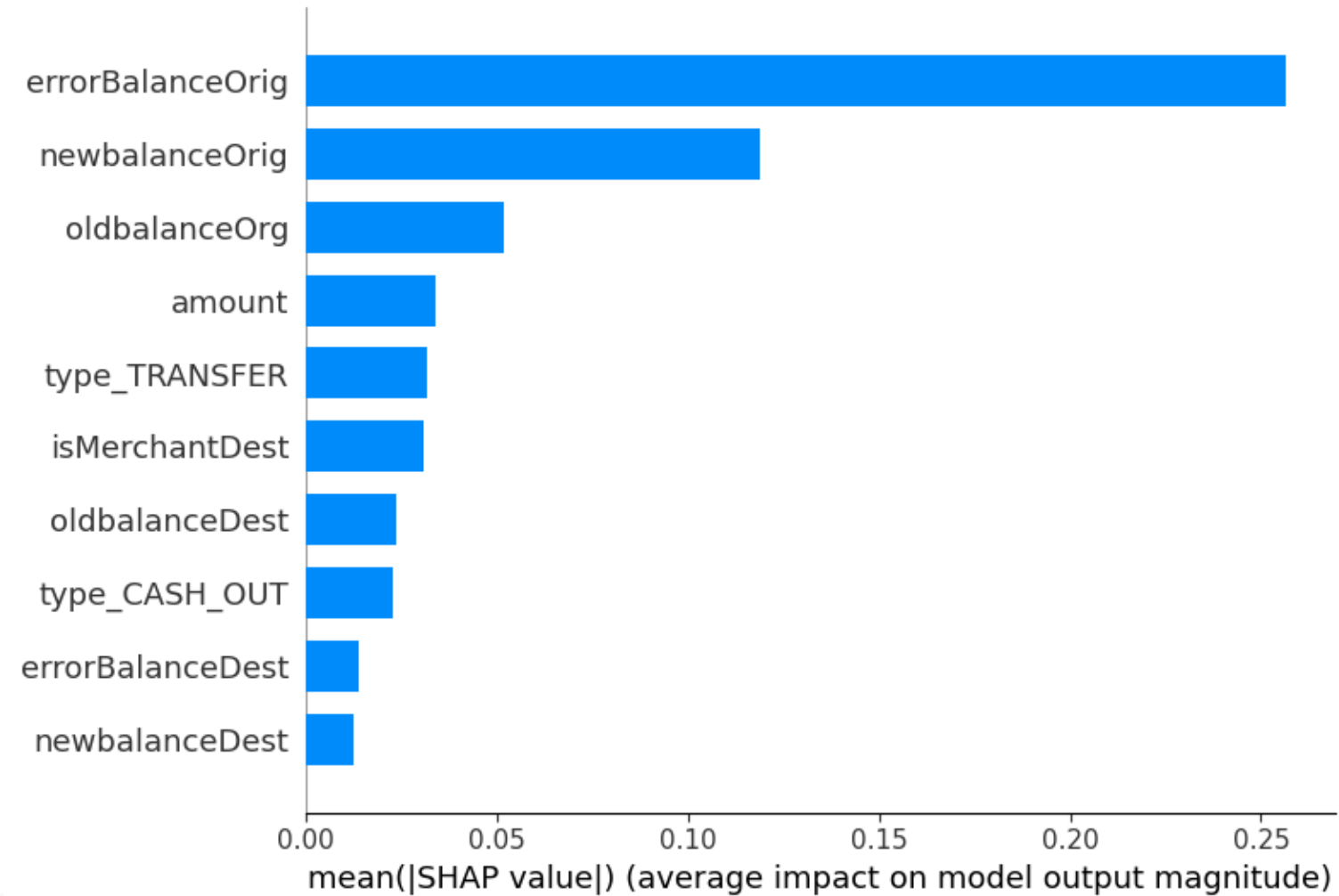
PR-AUC 0.126

This is caveat that despite the 0.99 PR-AUC, fraud detection at this point should not be tagged to determine fraud detection accuracy



Explainability and Fairness Finding

SHAP



Recall by Account Balance Tiering

Lowest balance **30%**

2nd Quartile **99%**

3rd Quartile **100%**

Highest Balance **100%**

- *Precision is 1.0 in all tiers; lowest balance quartile may be under-protected*



Limitations & Next Steps

Limitations, constraints

■ **Recall disparity between account segmentation** to be resolved

■ **No customer demographics** presented in the synthetic dataset

■ **Synthetic data** is simulated; not tested yet against live data

Next steps

- Secure necessary project approval for funding and personnel requests (working group, compliance and risk teams)
- Parallel run with existing rule-based fraud detection flagging against live data
- Investigate further fix for account balance segmentation disparity